Cofounder Vision

Show a workflow once. An agent takes it over, and checks every step with OpenCV 5.

OpenCV AI Competition 2026 (powered by AWS) · Technical report Team: Oren Chriqui (solo) · Featured path: Agentic Vision · Repository: https://github.com/chessoren/cofounder-vision Live endpoint: https://emmbbwhapidhsug3egofquuada0bqmel.lambda-url.us-west-2.on.aws/

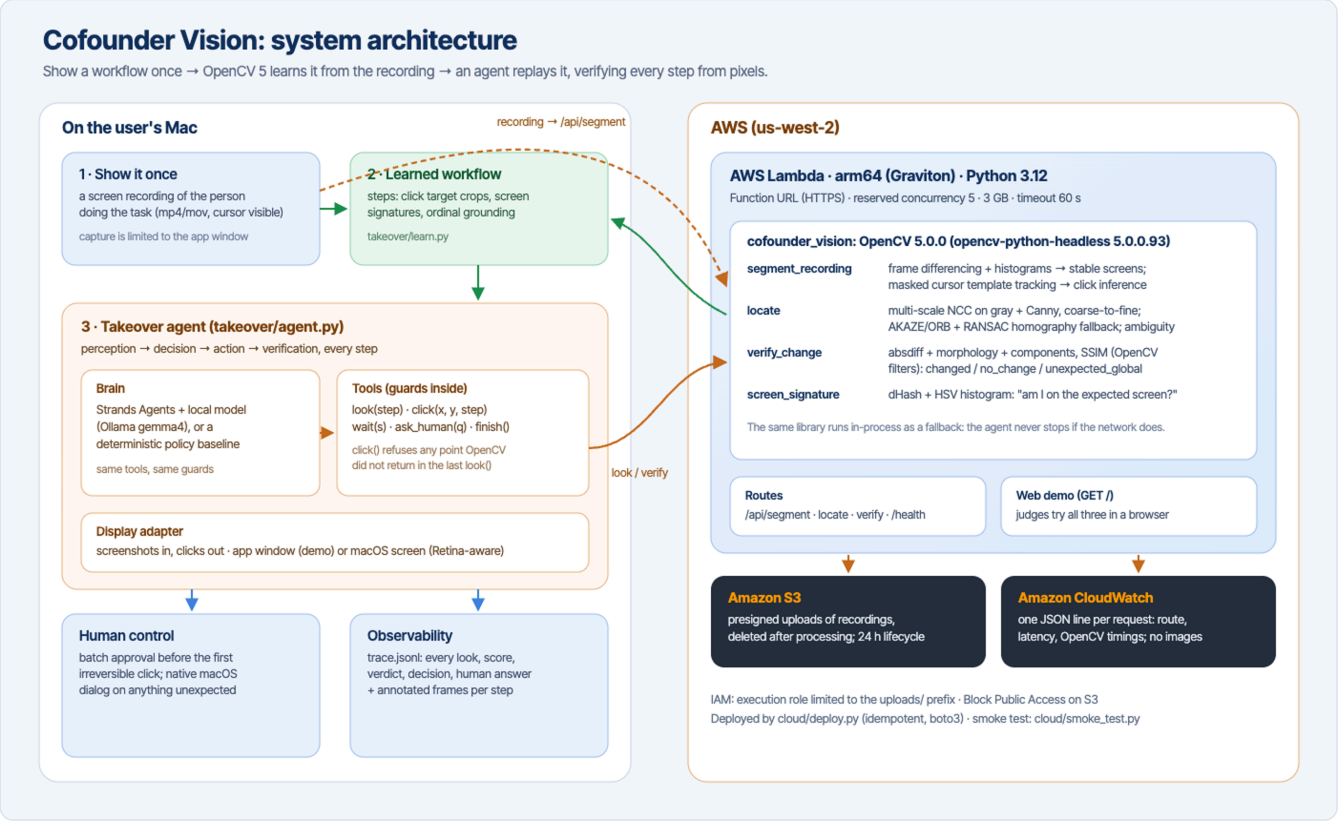
1. Problem and users

Office work is full of short, repetitive screen workflows. An accounts-payable clerk, for example, opens an invoice, matches it to its purchase order and approves it, thirty times a morning. Automation tools exist, but they start from a specification: *"what do you want to automate?"* Writing one takes longer than doing the task. Scripts that replay recorded click coordinates break as soon as a window moves, the display scale changes or a dialog appears, and they usually fail silently.

Users. Freelancers, bookkeepers, recruiters and operations staff. They live in a browser and a few SaaS tools, and no engineer will write automations for them.

Cofounder Vision asks for no specification. The person records the task once. OpenCV 5 turns the recording into steps, and an agent replays them on the live screen. Before every click it checks, with OpenCV, that it sees what the person saw. After every click it checks that the screen reacted the way it should. When the visual evidence says something is off, the agent changes what it does next: it waits, re-looks, or hands control back to the human.

2. System overview



The system has three parts:

1. **Learning.** `takeover/learn.py` uploads the recording to the vision service (S3 presigned PUT) and calls `/api/segment`. The result is a workflow: for each click, a crop of the clicked control, the full frame just before the click, and an *ordinal grounding* record. If the clicked control is one of several identical controls (a "Review" button on every row), the record says which one, counted top to bottom.
2. **Perception service** (`vision/`, deployed by `cloud/`). An OpenCV 5 library served by AWS Lambda on Graviton (arm64) behind a public Function URL, which also hosts the interactive web demo.
3. **Takeover agent** (`takeover/agent.py`). A perception → decision → action → verification loop with five tools. It is driven either by a Strands Agents LLM agent (local model through Ollama) or by a deterministic policy baseline that implements the same rules. The agent only receives pixels and can only act through those tools.

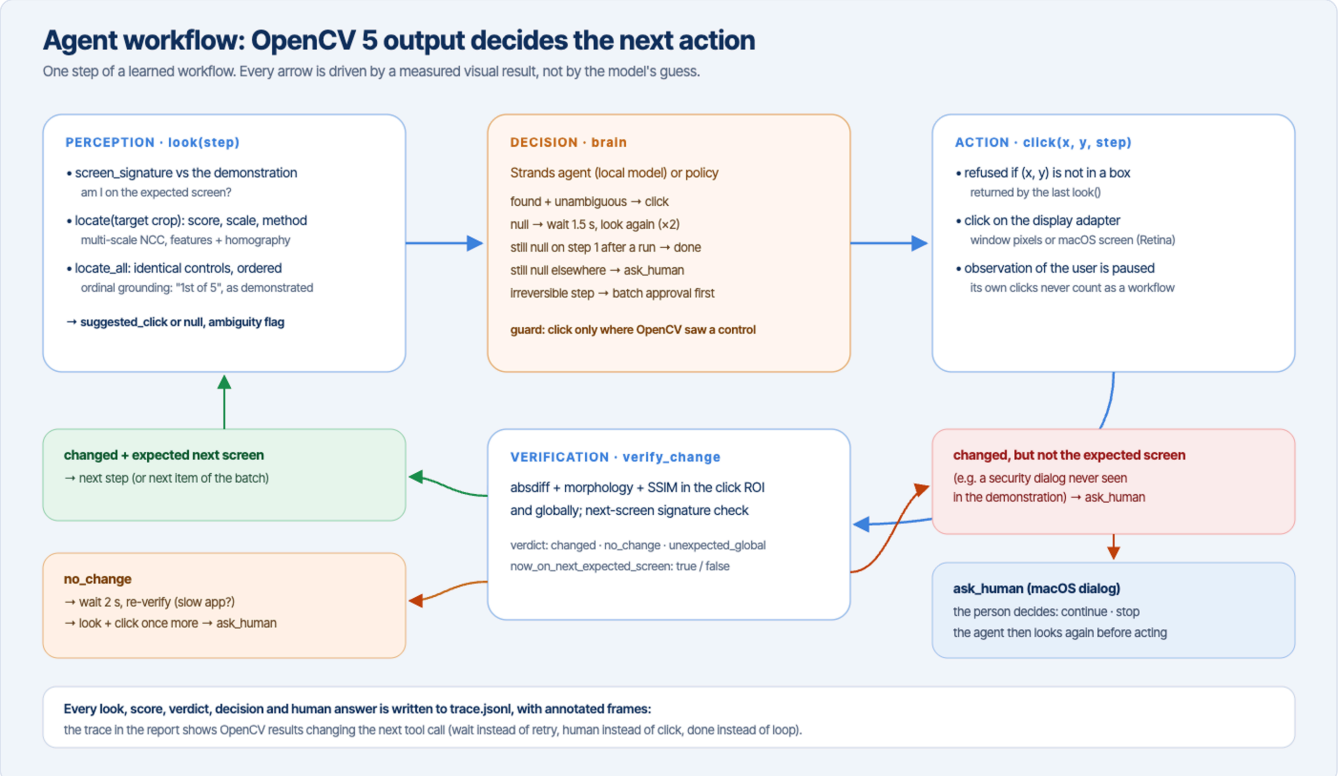
3. OpenCV 5 implementation

All perception uses **OpenCV 5.0.0** (`opencv-python-headless==5.0.0.93`, numpy pinned), with no contrib modules. The same code runs on Lambda arm64 and in-process; the local copy is the agent's fallback when the network is down.

Function	What it does	OpenCV 5 building blocks
`segment_recording`	recording → steps (click / type)	`VideoCapture`, frame `absdiff` ratio + HSV histogram correlation with hysteresis for stable screens; masked arrow-cursor template matching (`matchTemplate` with a mask) to track the cursor; click = dwell ≥ 150 ms followed by a local change near the cursor tip
`locate`	finds a learned control on a new screen	coarse-to-fine multi-scale `TM_CCOEFF_NORMED` on gray + Canny edges (scales 0.5–2.0); ambiguity test on the second peak after suppression; fallback ORB + ratio test + RANSAC `findHomography` (≥ 8 inliers; AKAZE is not in OpenCV 5 main modules)
`locate_all`	every identical control, ordered	multi-scale NCC + iterative non-maximum suppression
`verify_change`	did the action work?	`absdiff`, Gaussian blur, threshold, morphology, `connectedComponentsWithStats`; SSIM computed with OpenCV filters; verdict `changed` / `no_change` / `unexpected_global`
`screen_signature`	am I on the expected screen?	64-bit difference hash (`resize`) + HSV histogram (`calcHist`, `compareHist`)

Thresholds: `locate` accepts a match at NCC ≥ 0.80 by default; the agent requires ≥ 0.90 to act (see §6.3 for why).

4. The agentic loop (Agentic Vision path)



Tools. The agent has five tools:

Tool	What it does
`look(step)`	screen-signature check, `locate` + `locate_all`, suggested click
`click(x, y, step)`	the action, followed by `verify_change` and a next-screen signature check
`wait(s)`	pauses, then reports what changed since the last click
`ask_human(question)`	hands control to the person
`finish(status, why)`	ends the run

The visual evidence decides the next call:

- *Target not found:* wait and re-look, twice. If still nothing at step 1 after a completed run, the job is done; elsewhere the agent escalates to the human.
- `no_change` *after a click:* the agent waits and re-verifies before any retry, because the app may simply be slow. A retry happens once at most, then the agent escalates.
- *Changed, but not to the expected next screen* (a dialog never seen in the demonstration): the agent asks the human, then looks again before acting.

Guards in the tools, not in the prompt. `click()` refuses any coordinate that is not inside a control OpenCV returned in the last `look()`, so neither the model nor a bug can click somewhere unseen. Before the first irreversible action (*Approve*), the agent asks once for a batch approval. Every look, score, verdict, decision and human answer is written to `trace.jsonl`, together with an annotated frame.

Trace excerpt (scenario *slow*, policy brain, real run; `takeover/eval/slow-policy/trace.jsonl`). OpenCV says the click did nothing, so the agent waits instead of clicking again:

```
 t=11.958  click step=1  verdict=no_change  changed=False  local_change=0.0
t=15.801  wait 2.0 s    changed_since_last_click=True  verdict=changed  → continue, no second click
```

5. AWS deployment

Component	Configuration
AWS Lambda	`cofounder-vision`, **arm64 (Graviton)**, Python 3.12, 3008 MB, 60 s timeout, reserved concurrency 5 (abuse and cost guard), Function URL (HTTPS)
Package	built without Docker from pinned manylinux aarch64 wheels (`cloud/build.py`); `/health` reports the OpenCV version and CPU architecture at runtime
Amazon S3	temporary uploads through presigned PUT, Block Public Access, SSE-S3, objects deleted after processing plus a 24 h lifecycle rule
Amazon CloudWatch	one structured JSON log line per request (route, status, latency, OpenCV timings, sizes); no images, no IPs; 14-day retention
IAM	execution role = basic logging + access limited to the `uploads/` prefix of the bucket
Repeatability	`uv run --with boto3 python cloud/deploy.py` (idempotent create-or-update) and `cloud/smoke_test.py`

Server-side latency from CloudWatch: `/api/locate` 105–165 ms on 1280×800 and ≈ 550 ms on 2048×1400; `/api/verify` ≈ 75 ms; `/api/segment` ≈ 4 s for a 12.5 s recording; cold start 530–650 ms; peak memory ≈ 275 MB. Round trips from our client in Europe add 0.5–0.7 s, and the agent sends full PNG screenshots, so the agent's median perception round trip is 1.8–2.1 s (table 6.2). The smoke test passes 28/28 checks (all routes, validation errors, presigned upload, deletion after segmentation). *Not applied:* reserved concurrency 5 could not be set because this account's total Lambda concurrency limit is 10; the account limit is the effective cap.

6. Evaluation

6.1 Perception benchmark (`vision/eval/run_eval.py`)

Generated by `uv run python eval/run_eval.py` on 2026-10-26 22:46 (OpenCV 5.0.0, numpy 2.2.6, Python 3.12.14, Darwin arm64). Total runtime 39 s. Seed 20261026. All numbers below are computed by this run.

Images: 4 real screenshots of the Cofounder web UI (one page rendered at 1440x900, 1280x800, 900x1100 and 2048x1400 Retina 2x, captured with headless Chrome) + 3 synthetic app screens drawn with OpenCV (form, table, dashboard). Latencies are measured with `cv2.setNumThreads(1)` in worker processes (Apple Silicon); a 1-vCPU Lambda will differ.

1. locate()

48 element templates (sizes [26, 22] to [272, 70] px w,h) picked by Canny-contour proposals with filters (size, aspect, contrast, edge density, unique in their own image), x 10 conditions = 480 runs. Hit = found and centre error <= max(8 px, 10% of the template's larger side). Wrong = found but elsewhere (the dangerous failure). Negatives use a template from the other image family (real vs synthetic), so the element is absent. Template-only numbers are the same runs stopped after the template stage (its own timer).

Path: template

condition	n	hit	miss	wrong	ambiguous	via features	p50 ms	p95 ms
translate	48	100.0%	0.0%	0.0%	0.0%	0	197.6	252.9
scale_0.5	48	97.9%	2.1%	0.0%	4.3%	0	57.1	80.8
scale_0.8	48	100.0%	0.0%	0.0%	2.1%	0	138.2	176.3
scale_1.25	48	100.0%	0.0%	0.0%	0.0%	0	254.6	380.1
scale_2.0	48	100.0%	0.0%	0.0%	0.0%	0	275.4	373.5
brightness_contrast	48	95.8%	4.2%	0.0%	2.2%	0	193.7	240.3
blur_s1.5	48	31.2%	68.8%	0.0%	6.7%	0	198.6	259.5
jpeg_q40	48	100.0%	0.0%	0.0%	0.0%	0	192.6	249.7
occlusion_20	48	18.8%	62.5%	18.8%	16.7%	0	200.2	255.3
negative (absent)	48	-	-	FP 2.1%	0.0%	0	182.9	555.7

All positive conditions: hit 82.6%, wrong location 2.1%; negatives FP 2.1%.

Path: template+features

condition	n	hit	miss	wrong	ambiguous	via features	p50 ms	p95 ms
translate	48	100.0%	0.0%	0.0%	0.0%	0	197.7	252.9
scale_0.5	48	97.9%	2.1%	0.0%	4.3%	0	57.1	81.9
scale_0.8	48	100.0%	0.0%	0.0%	2.1%	0	138.2	176.3
scale_1.25	48	100.0%	0.0%	0.0%	0.0%	0	254.6	380.1
scale_2.0	48	100.0%	0.0%	0.0%	0.0%	0	275.5	373.5
brightness_contrast	48	95.8%	4.2%	0.0%	2.2%	0	193.7	240.3
blur_s1.5	48	31.2%	68.8%	0.0%	6.7%	0	222.0	279.6
jpeg_q40	48	100.0%	0.0%	0.0%	0.0%	0	192.6	249.7
occlusion_20	48	22.9%	56.2%	20.8%	16.7%	3	211.5	313.5
negative (absent)	48	-	-	FP 2.1%	0.0%	0	213.2	624.8

All positive conditions: hit 83.1%, wrong location 2.3%; negatives FP 2.1%.

Hit rate by template source (template+features, positive conditions):

source	templates	hit
cofounder_home_1440x900	8	80.6%
cofounder_home_retina_2048x1400	8	95.8%
cofounder_home_900x1100	8	79.2%
synth_form	8	80.6%
synth_table	8	80.6%
synth_dashboard	8	81.9%

Annotated failure cases: 60 images `eval/failures/locate_*.png` (green = ground truth, blue = prediction, template inset top-left).

2. verify_change()

168 before/after pairs over 7 screens. Positive = the screen really changed (button state, text inserted, navigation); negative = cursor moved only, caret blink only, JPEG re-encode (q40-75). Prediction 'changed' = verdict `changed` or `unexpected_global`.

	predicted changed	predicted no change
actually changed	84 (TP)	0 (FN)
actually unchanged	5 (FP)	79 (TN)

Accuracy **97.0%**. Latency p50 22.1 ms, p95 63.2 ms.

category	n	accuracy	verdicts
button_state	28	100.0%	changed: 28
text_insert	28	100.0%	changed: 28
navigation	28	100.0%	changed: 27, unexpected_global: 1
cursor_only	28	92.9%	changed: 2, no_change: 26
caret_blink	28	89.3%	changed: 3, no_change: 25
jpeg_noise	28	100.0%	no_change: 28

`unexpected_global` set (main area replaced by another screen, ROI in the untouched corner): 21/28 labelled `unexpected_global`; others: {'changed': 7}.

3. screen_signature() / compare_screens()

- Same screen after JPEG q50, 0.5x rescale, or cursor+typed text: 21/21 called `same` (min similarity 0.917).
- Different screens: 0/15 wrongly called `same` (max similarity 0.750).
- Same Cofounder page rendered at different window sizes/aspect ratios (layout reflows): 1/6 called `same` (reported separately: there is no single right answer).

4. segment_recording()

Four synthetic recordings (1280x800, 30 fps, mp4v, drawn arrow cursor on smooth eased paths, per-frame Gaussian noise sigma 2, blinking caret, idle periods) + one real recording of a demo app (1180x760, 10 fps, cursor drawn by the recorder, ground truth from its `.truth.json`). Analysis at 5 fps. A predicted step matches a ground-truth step if ldtl <= 0.6 s (one-to-one, greedy).

video	kind	GT steps	predicted	precision	recall	action acc.	click err px (mean/max)	time err s (mean/max)	runtime s
v1_form_typing	synthetic	5	5	100.0%	100.0%	100.0%	0.0 / 0.0	0.020 / 0.033	2.4
v2_cofounder_home	synthetic (real screenshot)	5	5	100.0%	100.0%	100.0%	0.0 / 0.0	0.040 / 0.067	2.8
v3_hard_dashboard	synthetic, hard	4	4	100.0%	100.0%	100.0%	0.0 / 0.0	0.075 / 0.100	2.1
v4_heldout_table	synthetic, held-out	4	5	80.0%	100.0%	100.0%	0.0 / 0.0	0.050 / 0.100	1.7
real_approve_invoice	real app recording	3	3	100.0%	100.0%	100.0%	4.2 / 5.4	0.120 / 0.150	2.4

Overall: 21 matched of 21 GT and 22 predicted steps -> precision 95.5%, recall 100.0%.

- **v4_heldout_table**: missed []; extra [('click', 0.1)] (see `failures/segment_v4_heldout_table.png`).

Development vs held-out. v1-v3 and the real demo-app recording were used while developing the segmenter (hover suppression, the relaxed stillness rule, chroma-aware change, click-time refinement and the moved-since-click fix were added after seeing failures on them), so their scores are optimistic. v4 was written after those rules were frozen and run once, without tuning.

The hard video (v3) contains: a hover highlight with a resting cursor and no click (must not become a step), two toggle clicks ~0.4 s apart (fast double action), and a click on a busy bar chart.

Notes on honesty / limits of this benchmark

- The cursor in the synthetic videos is drawn with the same renderer the detector's template is built from, so cursor detection is tested on the ideal shape (+ codec + noise). The real demo-app recording uses its own recorder-drawn

cursor.

- Perturbations are synthetic (resize, blur, JPEG, occluding rectangles); the Retina screenshot is a real 2x render, but templates are always cut from the same image they are searched in (before perturbation).
- Thresholds were set while writing the unit tests, before this benchmark was run; they were not tuned per condition.

6.2 Agent evaluation (`takeover/evaluate.py`)

The agent runs against **Ledgerly**, a small accounts-payable app built for this evaluation, under fault injection. Success is read from the app's own final state, not from the agent's claims. The demonstration recording is 12.5 s long and contains 3 clicks.

Scenario	What changes compared with the demonstration
clean	5 invoices left to approve
zoom125	the whole UI is rendered 25 % larger
slow	"Match PO" responds after 2.5 s
modal	a security dialog interrupts every approval
disabled	"Approve" never becomes enabled on one invoice

brain	scenario	expected	finish	approved (of 6)	clicks	no_change	unexpected	waits	human	refused	perception p50 (ms)	where	s
policy	clean	all 6 approved (5 by the agent), finish done	done	6	15	0	1	2	1	0	1902.85	aws-lambda-arm64	100.0
policy	zoom125	all approved despite 1.25x scale	done	6	15	0	1	2	1	0	1997.0	aws-lambda-arm64	104.5
policy	slow	waits on no_change instead of re-clicking; all approved	done	6	15	5	1	7	1	0	1992.9	aws-lambda-arm64	121.1
policy	modal	asks the human on every unexpected dialog; all approved	escalated	2	3	0	1	0	2	0	1996.75	aws-lambda-arm64	21.4
policy	disabled	approves 2, then stops and asks on INV-2043; no wrong approval	escalated	2	7	2	0	2	2	0	1888.6	aws-lambda-arm64	47.9
llm	clean	all 6 approved (5 by the agent), finish done	done	6	15	0	1	1	1	0	1896.2	aws-lambda-arm64	202.4
llm	zoom125	all approved despite 1.25x scale	done	6	15	0	1	1	1	0	2083.0	aws-lambda-arm64	201.8
llm	slow	waits on no_change instead of re-clicking; all approved	escalated	4	9	3	0	3	2	0	1784.9	aws-lambda-arm64	138.7
llm	modal	asks the human on every unexpected dialog; all approved	escalated	1	3	0	1	0	2	0	1781.6	aws-lambda-arm64	38.5
llm	disabled	approves 2, then stops and asks on INV-2043; no wrong approval	done	2	6	1	0	2	1	0	1910.7	aws-lambda-arm64	113.4

Reading the table. These are the final run (10 runs, all perception on AWS Lambda arm64). - **Policy brain.** Clean, 125 % zoom and the slow app: all invoices approved, with no wrong click. In the slow scenario the 5 `no_change` verdicts led to 7 waits and no double click. The disabled button: the agent stopped and asked after 2 approvals, which is the correct outcome. - **Policy brain, dialog scenario.** In this final run the agent escalated after INV-2042 (2 of 6), so it did not finish the batch. An earlier run (lower acting threshold, 0.85) approved 6 of 6 with 6 human turns. That threshold was raised after the failure described in §6.3, and we report the final run as measured. - **LLM brain** (Strands Agents + local gemma4:e2b). Clean and zoom: 6 of 6. Slow: escalated after 4. Dialog: escalated after 1. Disabled: no wrong approval, but it reported "done" instead of "escalated". - **Guards and safety.** No refused clicks were needed in these runs; the guard did fire in development (§6.3). No run approved a wrong invoice.

6.3 Failure cases

- **A dialog button mistaken for the next step's target** (found during development, fixed). After *Approve*, a security dialog appeared, and its blue "I'm here" button matched the "Review" template at NCC 0.86. The agent believed it was back on the inbox and clicked the dialog. Fixes: (1) each step stores the global change its click caused in the demonstration, and an effect more than 3× larger (+0.1) counts as *unexpected* and goes to the human; (2) the acting threshold was raised from 0.85 to 0.90.
- **Look-alike buttons** (development). At NCC 0.83, the "Review" template matched the green "Approve" button. That is another reason for the 0.90 acting threshold.
- **The small LLM loses track over long horizons** (development). As a single 15-step conversation, gemma4:e2b looped on `look(0)` and tried a coordinate OpenCV had not returned; `click()` refused it. Per-step conversations with a 10-call budget fixed the clean case. Slow and dialog runs still fail (table 6.2).
- **Perception limits** (benchmark §6.1). Blur σ = 1.5 drops locate hits to 31 %, and 20 % occlusion drops them to 23 %, with 21 % wrong locations. Caret blink and cursor-only changes cause 5 false "changed" verdicts out of 56. The held-out recording gets 1 extra click step (precision 80 %). Annotated failure images are in `vision/eval/failures/`.

7. Limitations

- **Click-based workflows only.** Typed text is detected as a "type" step, but the agent does not yet know *what* to type for a new item: that needs OCR or a vision-language model.
- **One demonstration per workflow.** Controls whose appearance changes a lot between items (dynamic labels) defeat template matching; the features fallback helps only on textured controls.
- **Interpreting surprises is delegated.** The agent detects unexpected screens reliably, but it does not interpret them: it asks the human. That is a deliberate choice, but it limits autonomy.
- **Synthetic demonstration.** The demonstration in the evaluation is performed by a scripted demonstrator and recorded from the app window with a recorder-drawn cursor. Real macOS screen recordings are supported (the masked cursor tracking also handles the system cursor), but they are not part of the benchmark.
- **The language model is local.** The LLM brain runs a small local model (Ollama) because Amazon Bedrock model access was not available on our AWS account during the competition. The deterministic policy implements the same rules and is the evaluation baseline.

8. Responsible use

- **Privacy.** Only the target app's window is captured. The agent never sees other apps, notifications or the desktop. Uploads are deleted after processing; nothing is stored or used for training. Logs contain no image data.
- **Human control.** The batch approval protects irreversible actions. Anything unexpected goes to the human, never to a guess. Stop is always available, and the human can also stop the agent by moving the mouse to a screen corner (pyautogui fail-safe) on the real-screen adapter.
- **No invented clicks.** Coordinates must come from OpenCV detections, as enforced in `click()`.
- **Honest reporting.** Every number in this report is produced by the scripts in the repository and can be regenerated.

9. Reproduce

```
 git clone https://github.com/chessoren/cofounder-vision && cd cofounder-vision
cd vision && uv sync && uv run python -m pytest -q && uv run python eval/run_eval.py          # perception
benchmark
cd ../cloud && uv run --with boto3 python deploy.py && uv run --with boto3 python smoke_test.py # AWS (your
account)
npx electron ../demo-app/window.cjs &                                                         # the target app
cd ../takeover && uv sync && uv run python learn.py ../demo-app/recordings/approve-invoice.mp4
uv run python evaluate.py --brains policy,llm                                                 # agent
evaluation
```

10. Disclosure

All the code in this repository was written during the competition's build phase:

- the OpenCV 5 library (`vision/`);
- the AWS Lambda service and web demo (`cloud/`);
- the learning step and the takeover agent (`takeover/`);
- the evaluation app and harness (`demo-app/`, `takeover/evaluate.py`).

AI coding assistants were used during development. The agentic workflow evaluated here is the product's runtime loop, not the use of a coding assistant.